

# KrysukEngine

Kodowa nazwa projektu: Krysuk

Temat projektu: Lekki, przenośny silnik gier 3D z architekturą ECS

## Skład zespołu

Karyna Rudenko – *team leader*

Viktor Zasimovych – *członek zespołu*

15 kwietnia 2026

[Repozytorium GitHub](#)

# 1 Wizja systemu

**KrysukEngine** to aplikacja desktopowa przeznaczona dla deweloperów gier komputerowych. System umożliwia tworzenie gier w oparciu o obiektowy silnik z paradygmatem **ECS (Entity-Component-System)**. Dostarcza wbudowany render oraz narzędzia ułatwiające integrację zasobów. Zapewnia przenośność między platformami (Linux, Windows). Dla graczy gwarantuje wysoką wydajność i umożliwia granie na różnych platformach.

## Główne korzyści

- Oszczędność czasu programistów – brak potrzeby tworzenia silnika od podstaw.
- Niższe koszty produkcji i utrzymania projektu.
- System ECS pozwala na elastyczny i niezależny rozwój komponentów.
- Zintegrowany render skraca czas implementacji grafiki.
- Wsparcie popularnych formatów modeli 3D i tekstur.
- Łatwiejsze utrzymanie kodu dzięki modularnej architekturze.
- Możliwość szybkiego portowania gry na różne platformy.

## 2 Zakres usług

W ramach projektu zostanie zrealizowane:

- Implementacja podstawowego silnika ECS (Entity-Component-System).
- Wbudowany renderer 3D z obsługą podstawowych shaderów.
- Obsługa tekstur (PNG, JPG).
- System wejścia (klawiatura, mysz).
- Aplikacja desktopowa działająca na Windows i Linux.

### Ograniczenia – czego nie będzie w projekcie

- Edytora wizualnego (scene editor) w wersji 1.0.
- Zaawansowanego systemu fizyki (np. detekcja kolizji będzie ograniczona).
- Silnika sieciowego (multiplayer).
- Wsparcia dla konsol (PlayStation, Xbox, Switch).
- Dźwięku i zaawansowanego audio.
- Animacji szkieletowych.
- Silnika AI / ścieżkowego.

### 3 Architektura logiczna

System opiera się na modularnej architekturze, która umożliwia niezależny rozwój komponentów. Poniżej przedstawiono główne elementy składowe oraz ich komunikację.

#### Opis elementów

- **Aplikacja desktopowa** – warstwa zarządzająca cyklem życia aplikacji (okno, pętla główna).
- **ECS Core** – serce silnika: zarządza encjami, komponentami i systemami.
- **Renderer 3D** – odpowiedzialny za rysowanie obiektów 3D na ekranie.
- **Manager assetów** – ładuje i przechowuje modele, tekstury, shadery.
- **System wejścia** – przetwarza zdarzenia z klawiatury i myszy.

#### Komunikacja

- Pętla główna aplikacji wywołuje kolejno: aktualizację ECS, przetworzenie wejścia, a następnie rendering.
- ECS Core modyfikuje dane komponentów, które są odczytywane przez renderer.
- System wejścia modyfikuje komponenty (np. pozycja kamery, ruch encji).
- Manager assetów dostarcza zasoby do renderera na żądanie.

## 4 Główne funkcje – Epiki i Historyjki użytkownika

Na podstawie potrzeb klienta oraz zakresu usług zdefiniowano następujące epiki (grupy funkcji) i przypisane do nich historyjki użytkownika.

### Epik 1: Podstawowy silnik ECS

- **H1:** Jako deweloper chcę tworzyć encje i dodawać do nich komponenty, aby móc budować obiekty gry.
- **H2:** Jako deweloper chcę definiować własne systemy, które przetwarzają encje z określonymi komponentami.
- **H3:** Jako deweloper chcę, aby silnik automatycznie zarządzał cyklem życia systemów (init, update, destroy).

### Epik 2: Renderer 3D

- **H4:** Jako deweloper chcę rysować modele 3D na ekranie z obsługą podstawowych shaderów.
- **H5:** Jako deweloper chcę ładować tekstury i nakładać je na modele.
- **H6:** Jako deweloper chcę sterować kamerą (obracanie, przybliżanie), aby testować scenę z różnych perspektyw.

### Epik 3: Obsługa assetów

- **H7:** Jako deweloper chcę ładować modele 3D z plików OBJ, aby używać własnych zasobów.
- **H8:** Jako deweloper chcę ładować tekstury z plików PNG/JPG.

### Epik 4: Przenośność między platformami

- **H9:** Jako deweloper chcę uruchomić tę samą grę na Windows i Linux bez zmian w kodzie.
- **H10:** Jako gracz chcę grać w gry stworzone w Vector3D na moim systemie operacyjnym (Windows/Linux).